

SCJP 1.6 (CX-310-065 , CX-310-066)

Subject: Assignments

Wrapper classes, autoboxing, unboxing , garbage collection

Total Questions : 31

Prepared by : <http://www.javacertifications.net>

SCJP 6.0: Assignment

Questions Question - 1

What is the output for the below code ?

```
1. public class Test {  
2.     public static void main(String[] args){  
3.         int i = 010;  
4.         int j = 07;  
5.         System.out.println(i);  
6.         System.out.println(j);  
7.     }  
8. }
```

1.8 7

2.10 7

3.Compilation fails with an error at line 3

4.Compilation fails with an error at line 5

Explanation :

A is the correct answer.

By placing a zero in front of the number is an integer in octal form. 010 is in octal form so its value is 8.

Question - 2

What is the output for the below code ?

```
1. public class Test {  
2.     public static void main(String[] args){  
3.         int i = 0X10;  
4.         int j = 7;  
5.         System.out.println(i);  
6.         System.out.println(j);  
7.     }  
8. }
```

1.16 7

2.10 7

3.Compilation fails with an error at line 3

4.Compilation fails with an error at line 5

Explanation :

A is the correct answer.

By placing a 0X in front of the number is an integer in hexadecimal form. 0X10 is in hexadecimal form so its value is 16.

Question - 3

What is the output for the below code ?

```
1. public class Test {  
2. public static void main(String[] args){  
3. byte i = 2;  
4. byte j = 7;  
5. byte k = i+j;  
6. System.out.println(k);  
7. }  
8. }
```

1.9

2.0

3.Compilation fails with an error at line 5

4.Compilation fails with an error at line 6

Explanation :

C is the correct answer.

int or smaller expressions always resulting in an int. So compiler complain about Type mismatch: cannot convert from int to byte.

Question - 4

What is the output for the below code ?

```
1. public class Test {  
2. public static void main(String[] args){  
3. byte i = 128;  
4. System.out.println(i);  
5. }  
6. }
```

1.128

2.0

3.Compilation fails with an error at line 3

4.Compilation fails with an error at line 4

Explanation :

C is the correct answer.

byte can only hold up to 127. So compiler complain about possible loss of precision.

Question - 5

What is the output for the below code ?

```
1. public class Test {  
2.     public static void main(String[] args) {  
3.         int i = 856;  
4.         long k = i;  
5.         System.out.println(k);  
6.     }  
7. }
```

- 1. 856
- 2. 0
- 3. Compilation fails with an error at line 4
- 4. Compilation fails with an error at line 5

Explanation :

C is the correct answer.

Implicit cast, an int value always fits in a long because long has big container.

Question - 6

What is the output for the below code ?

```
1. public class Test {  
2.     public static void main(String[] args) {  
3.         byte b = 6;  
4.         b+=8;  
5.         System.out.println(b);  
6.         b = b+7;  
7.         System.out.println(b);  
8.     }  
9. }
```

- 1. 14 21
- 2. 14 13
- 3. Compilation fails with an error at line 6
- 4. Compilation fails with an error at line 4

Explanation :

C is the correct answer.

int or smaller expressions always resulting in an int. So compiler complain about Type mismatch: cannot convert from int to byte for `b = b+7`; But `b += 7`; // No problem because `+=`, `-=`, `*=`, and `/=` will all put in an implicit cast. `b += 7` is same as `b = (byte)b+7` so compiler not complain.

Question - 7

What is the output for the below code ?

```
1. public class Test {
2.     int i=8;
3.     int j=9;
4.     public static void main(String[] args){
5.         add();
6.     }
7.     public static void add(){
8.         int k = i+j;
9.         System.out.println(k);
10.    }
11. }
```

1.17

2.0

3.Compilation fails with an error at line 5

4.Compilation fails with an error at line 8

Explanation :

D is the correct answer.

i and j are instance variable and attempting to access an instance variable from a static method. So Compilation fails .

Question - 8

What is the output for the below code ?

```
1. public class Test {
2.     public static void main(String[] args){
3.         for(int i=0;i<5;i++){
4.             boolean isTrue = false;
5.             if(i == 5){
6.                 isTrue = true;
7.             }
8.         }
9.         System.out.println(isTrue);
10.    }
11. }
```

1.false

2.true

3.Compilation fails with an error at line 5

4.Compilation fails with an error at line 9

Explanation :

D is the correct answer.

isTrue variable is only visible within for loop. Attempting to use a block variable after the code block has completed. So Compilation fails .

Question - 9

What is the output for the below code ?

```
public class Test {  
    public static void main(String[] args){  
        String value = "abc";  
        changeValue(value);  
        System.out.println(value);  
    }  
  
    public static void changeValue(String a){  
        a = "xyz";  
    }  
}
```

- 1.abc
- 2.xyz
- 3.Compilation fails
- 4.Compilation clean but no output

Explanation :

A is the correct answer.

Java pass reference as value. passing the object reference, and not the actual object itself. Simply reassigning to the parameter used to pass the value into the method will do nothing, because the parameter is essentially a local variable.

Question - 10

What is the output for the below code ?

```
public class Test {  
    public static void main(String[] args){  
        StringBuffer stb = new StringBuffer( "Hello" );  
        changeValue(stb);  
        System.out.println(stb);  
    }  
  
    public static void changeValue(StringBuffer a){  
        a.append(" man");  
    }  
}
```

- 1.Hello man

- 2.Hello
- 3. man
- 4.Compilation clean but no output

Explanation :

A is the correct answer.

Java pass reference as value. passing the object reference, and not the actual object itself. In this case value is modified because object passed to a method, modified that object's internal state.

Question - 11

What is the output for the below code ?

```
public class Test {  
    public static void main(String[] args) {  
  
        Integer X = new Integer(12);  
        increate(X);  
        System.out.println(X);  
    }  
  
    public static void increate(Integer N) {  
        N = new Integer(N.intValue() + 1);  
    }  
}
```

- 1.12
- 2.13
- 3.0
- 4.Compilation clean but no output

Explanation :

A is the correct answer.

Java pass reference as value. passing the object reference, and not the actual object itself. Simply reassigning to the parameter used to pass the value into the method will do nothing, because the parameter is essentially a local variable.

Question - 12

What is the output for the below code ?

```
public class Test {  
    static Integer X ;  
  
    public static void main(String[] args) {  
  
        increate(X);  
    }  
}
```

```

        System.out.println(X);
    }

    public static void increate(Integer N) {
        N = new Integer(N.intValue() + 1);
    }
}

```

- 1.0
- 2.1
- 3.Compilation fails
- 4.Compilation clean but throws NullPointerException

Explanation :

D is the correct answer.

N.intValue() throws a NullPointerException because value of X is null. static Integer X ; is not initialize so value of X is null.

Question - 13

What is the output for the below code ?

```

public class Test {
    static Integer X ;
    public static void main(String... args) {
        increate(X);
        System.out.println(X);
    }
    public static void increate(int N) {
        int k = N +1;
        System.out.println(k);
    }
}

```

- 1.1 0
- 2.0 1
- 3.Compilation fails
- 4.Compilation clean but throws NullPointerException

Explanation :

D is the correct answer.

During unbox (public static void increate(int N)) throws a NullPointerException because value of X is null. static Integer X ; is not initialize so value of X is null.

Question - 14

What is the output for the below code ?

```

public class Test {

    public static void printValue(int i){
        System.out.println("int");
    }
    public static void printValue(long i){
        System.out.println("long");
    }

    public static void main(String... args) {
        byte b = 9;
        short s = 3;
        long l = 5;
        printValue(b);
        printValue(s);
        printValue(l);
    }
}

```

- 1.long
- 2.int long
- 3.int int long
- 4.Compilation clean but throws RuntimeException

Explanation :

C is the correct answer.

byte and the short arguments are not available so byte and the short arguments are implicitly widened to match the version of the printValue(int i). Primitive widening uses the smallest method argument possible. (For Example if you pass short value to a method but method with short argument is not available then compiler choose method with int argument). long uses the long version of printValue() method.

Question - 15

What is the output for the below code ?

```

public class Test {

    public static void printValue(Integer i){
        System.out.println("int");
    }
    public static void printValue(long i){
        System.out.println("long");
    }

    public static void main(String... args) {
        int b = 9;
        printValue(b);
    }
}

```

- 1.long
- 2.int

- 3.Compilation fails
- 4.Compilation clean but throws RuntimeException

Explanation :

A is the correct answer.

Primitive widening uses the smallest method argument possible. (For Example if you pass short value to a method but method with short argument is not available then compiler choose method with int argument). But widening a primitive parameter is more desirable than performing an autoboxing. So compiler chooses printValue(long i).

Question - 16

What is the output for the below code ?

```
public class Test {  
  
    public static void printValue(int i, int j, int k){  
        System.out.println("int");  
    }  
    public static void printValue(byte...b){  
        System.out.println("long");  
    }  
  
    public static void main(String... args) {  
        byte b = 9;  
        printValue(b,b,b);  
    }  
}
```

- 1.long
- 2.int
- 3.Compilation fails
- 4.Compilation clean but throws RuntimeException

Explanation :

B is the correct answer.

Primitive widening uses the smallest method argument possible. (For Example if you pass short value to a method but method with short argument is not available then compiler choose method with int argument). But in this case compiler will prefer the older style before it chooses the newer style, to keep existing code more robust. var-args method is looser than widen.

Question - 17

What is the output for the below code ?

```
public class Test {
```

```

    public static void printValue(byte i, byte j, byte k){
        System.out.println("byte123");
    }
    public static void printValue(byte...b){
        System.out.println("byte...");
    }

    public static void main(String... args) {
        byte b = 9;
        printValue(b,b,b);
    }
}

```

- 1.byte...
- 2.byte123
- 3.Compilation fails
- 4.Compilation clean but throws RuntimeException

Explanation :

B is the correct answer.

var-args method is looser than old style.

Question - 18

What is the output for the below code ?

```

1. public class Test {
2.     public static void printValue(Long l){
3.         System.out.println("Long");
4.     }
5.     public static void main(String... args) {
6.         printValue(new Integer(5));
7.     }
8. }

```

- 1.Long
- 2.Compilation fails with an error at line 6
- 3.Compilation fails with an error at line 2
- 4.Compilation clean but throws RuntimeException

Explanation :

B is the correct answer.

wrapper classes will not widen from one to another so compiler complain.

Question - 19

What is output of the code bellow ?

```
public class Test {  
  
    public static void main(String[] args) {  
        List < Integer > list = new ArrayList < Integer >();  
        list.add(0, 59);  
        int total = list.get(0);  
        System.out.println(total);  
    }  
  
}
```

Options are

- A.Compile time error, because you have to do int total = ((Integer) (list.get(0))).intValue();
- B.59
- C.Runtime Exceptions
- D.None of the above

Answer : B

Explanation :

B is the correct answer.

Manual conversion between primitive types (such as an int) and wrapper classes (such as Integer) is necessary when adding a primitive data type to a collection in jdk1.4 but The new autoboxing/unboxing feature eliminates this manual conversion in jdk 1.5

Question - 20

What is the output of the bellow code?

```
public class Test {  
  
    public static void main(String[] args) {  
        Boolean expr = true;  
        if (expr) {  
            System.out.println("true");  
        } else {  
            System.out.println("false");  
        }  
    }  
  
}
```

Options are

- A.true
- B.Compile with error because you can't use Boolean object in if()
- C.Runtime Exception
- D.None of the above

Answer : A

Explanation :

A is the correct answer.

In the if statement, condition can be Boolean in jdk1.5

Question - 21

Is the bellow code compile without error ?

```
public class Test {  
  
    public static void main(String[] args) {  
        Boolean expr = true;  
        while (expr) {  
            expr = !expr;  
        }  
    }  
  
}
```

Options are

- A.No, while condition can't allow Boolean object, it allows boolean primitive type
- B.Yes, while condition allow Boolean object
- C.Can't say
- D.None of the above

Answer : B

Explanation :

B is the correct answer.

In the while, do-while and for statements, the condition can be Boolean

Question - 22

What is the output for the bellow code ?

```
public class Test {
```

```

        public static void main(String[] args) {
            String[] words = new String[] { "aaa", "bbb", "ccc",
"aaa"};
            Map < String, Integer > m = new TreeMap < String,
Integer >();
            for (String word : words) {
                Integer freq = m.get(word);
                m.put(word, freq == null ? 1 : freq + 1);
            }
            System.out.println(m);
        }
    }
}

```

Options are

- A. {aaa=2, bbb=1, ccc=1}
- B. Compile with Error
- C. Runtime Exception
- D. None of the above

Answer : A

Explanation :

A is the correct answer.

Boxing and unboxing in collections

Question - 23

```

ArrayList list = new ArrayList();
list.add(0, new Integer(42));
int total = (list.get(0)).intValue();

```

is same as in jdk1.5 is?

Options are

- A. ArrayList list = new ArrayList(); list.add(0, 42); int total = list.get(0);
- B. ArrayList list = new ArrayList(); list.add(0, 42); int total = (int)list.get(0);
- C. ArrayList list = new ArrayList(); list.add(0, 42); int total = list.get(0).int();
- D. None of the above

Answer : A

Explanation :

A is the correct answer.

In JDK 1.5 Don't need to cast

Question - 24

What is the output of the bellow code ?

```
public class Test {  
  
    public static void main(String[] args) {  
        Integer in = new Integer(null);  
        System.out.println(in.intValue());  
    }  
  
}
```

Options are

- A.0
- B.Compile with error because , Integer in = new Integer(null);
- C.java.lang.NumberFormatException
- D.None of the above

Answer : C

Explanation :

C is the correct answer.

An Integer expression can have a null value. If your program tries to unbox null, it will throw a NullPointerException

Question - 25

What is the output of bellow code ?

```
public class Test {  
  
    public static void main(String[] args) {  
        Integer i5 = new Integer(5);  
        Integer j5 = new Integer(5);  
        if(i5 == j5){  
            System.out.println("true");  
        }else{  
            System.out.println("false");  
        }  
    }  
  
}
```

Options are

- A.true
- B.false
- C.Compile with error
- D.None of the above

Answer : B

Explanation :

B is the correct answer.

checks references , so both are different

Question - 26

What is the output of the bellow code?

```
public class Test {  
  
    public static void main(String[] args) {  
        Integer i5 = new Integer(5);  
        Integer j5 = new Integer(5);  
        if(i5.equals(j5)){  
            System.out.println("equal");  
        }else{  
            System.out.println("false");  
        }  
    }  
}
```

Options are

- A.equal
- B.false
- C.Compile with error
- D.None of the above

Answer : A

Explanation :

A is the correct answer.

Question - 27

What is the output?

```
public class Test {
```

```
        public static void main(String[] args) {
            Integer i5 = 2;
            Integer j5 = 2;
            System.out.println(i5 == j5);
            if(i5 == j5){
                System.out.println("equal");
            }else{
                System.out.println("not equal");
            }
        }
    }
}
```

Options are

- A.false not equal
- B.true not equal
- C.true equal
- D.None of the above

Answer : A

Explanation :

A is the correct answer.

Question - 28

What is the output for the below code?

```
public class Test {

    public static void main(String[] args) {

        Short expr = 1;
        switch (expr) {
            case 1:
                System.out.println("ONE"); break;
            case 2:
                System.out.println("TWO"); break;
            case 3:
                System.out.println("THREE"); break;
            default:
                assert false;
        }
    }
}
```

Options are

- A.ONE

- B.Compile With error because you can't use sort in switch.
- C.Runtime Exceptions
- D.None of the above

Answer : A

Explanation :

A is the correct answer.

In the switch statement, the switch expression can be Character, Byte, Short or Integer in JDK 1.5

Question - 29

What is the output for the below code?

```
public class Test {  
  
    public static void main(String[] args) {  
  
        Integer i = null;  
        int j = i;  
        System.out.println(j);  
    }  
  
}
```

Options are

- A.0
- B.null
- C.Compile with error
- D.java.lang.NullPointerException

Answer : D

Explanation :

D is the correct answer.

trying to unbox null , So throws NullPointerException

Question - 30

What is the output of the bellow code ?

```
public class Test {  
  
    public static void main(String[] args) {
```

```

        doSomething(1);
        doSomething(2.0);

    }

    public static void doSomething(double num) {
        System.out.println("double : " + num);
    }

    public static void doSomething(Integer num) {
        System.out.println("Integer : " + num);
    }

}

```

Options are

- A.double : 1.0 double : 2.0
- B.Integer : 1 double : 2.0
- C.Integer : 1 Integer : 2
- D.None of the above

Answer : A

Explanation :

A is the correct answer.

widening a primitive parameter is more desirable than performing an autoboxing.

Question - 31

What is the output ?

```

public class Test {

    public static void main(String[] args) {
        Integer i=1;
        doSomething(i);
        doSomething(2.0);

    }

    public static void doSomething(double num) {
        System.out.println("double : " + num);
    }

    public static void doSomething(Integer num) {
        System.out.println("Integer : " + num);
    }

}

```

Options are

- A.Integer : 1 double : 2.0
- B.double : 1.0 double : 2.0
- C.double : 1.0 Integer: 2
- D.None of the above

Answer : A

Explanation :

A is the correct answer.

Method **doSomething(Integer num)** match with Integer argument.